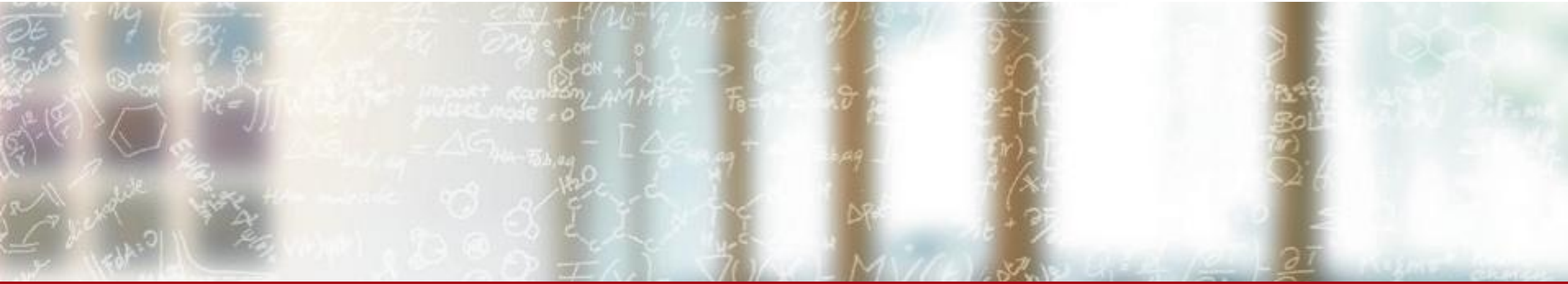




CSCS

Centro Svizzero di Calcolo Scientifico
Swiss National Supercomputing Centre

ETH zürich



HPC Containers on Alps

Alberto Madonna (ETHZ / CSCS)

CSCS User Lab Day
August 28th, 2026



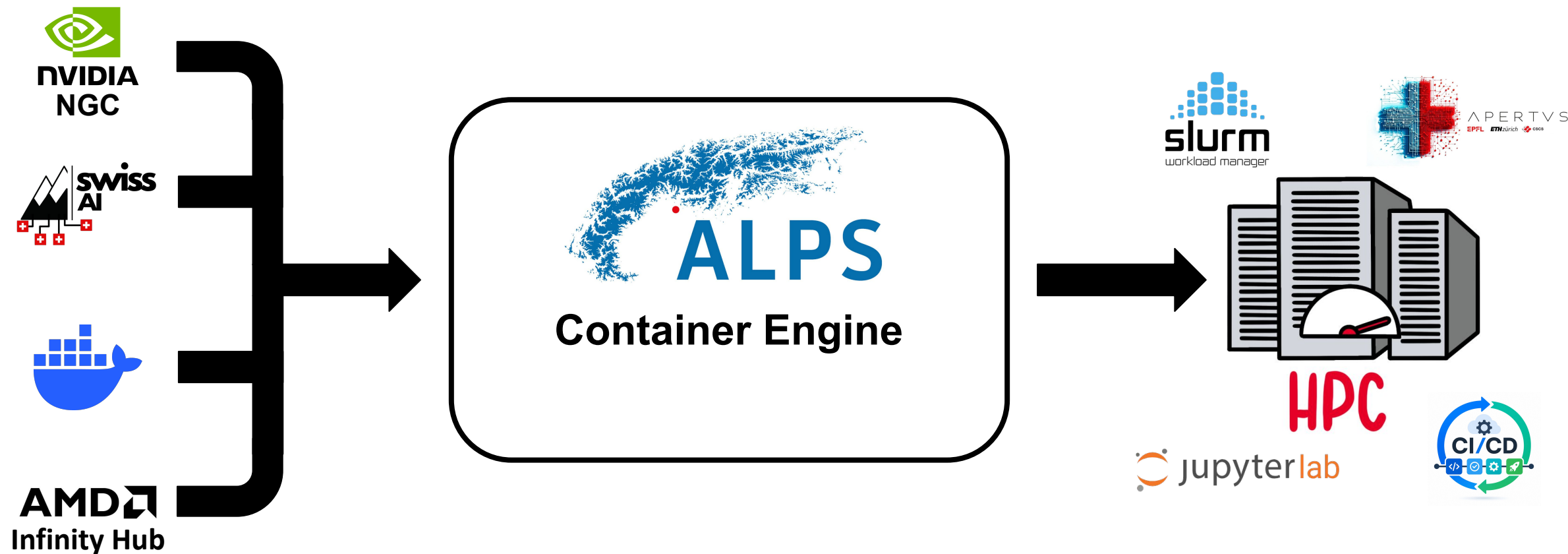
CSCS

Centro Svizzero di Calcolo Scientifico
Swiss National Supercomputing Centre

ETH zürich

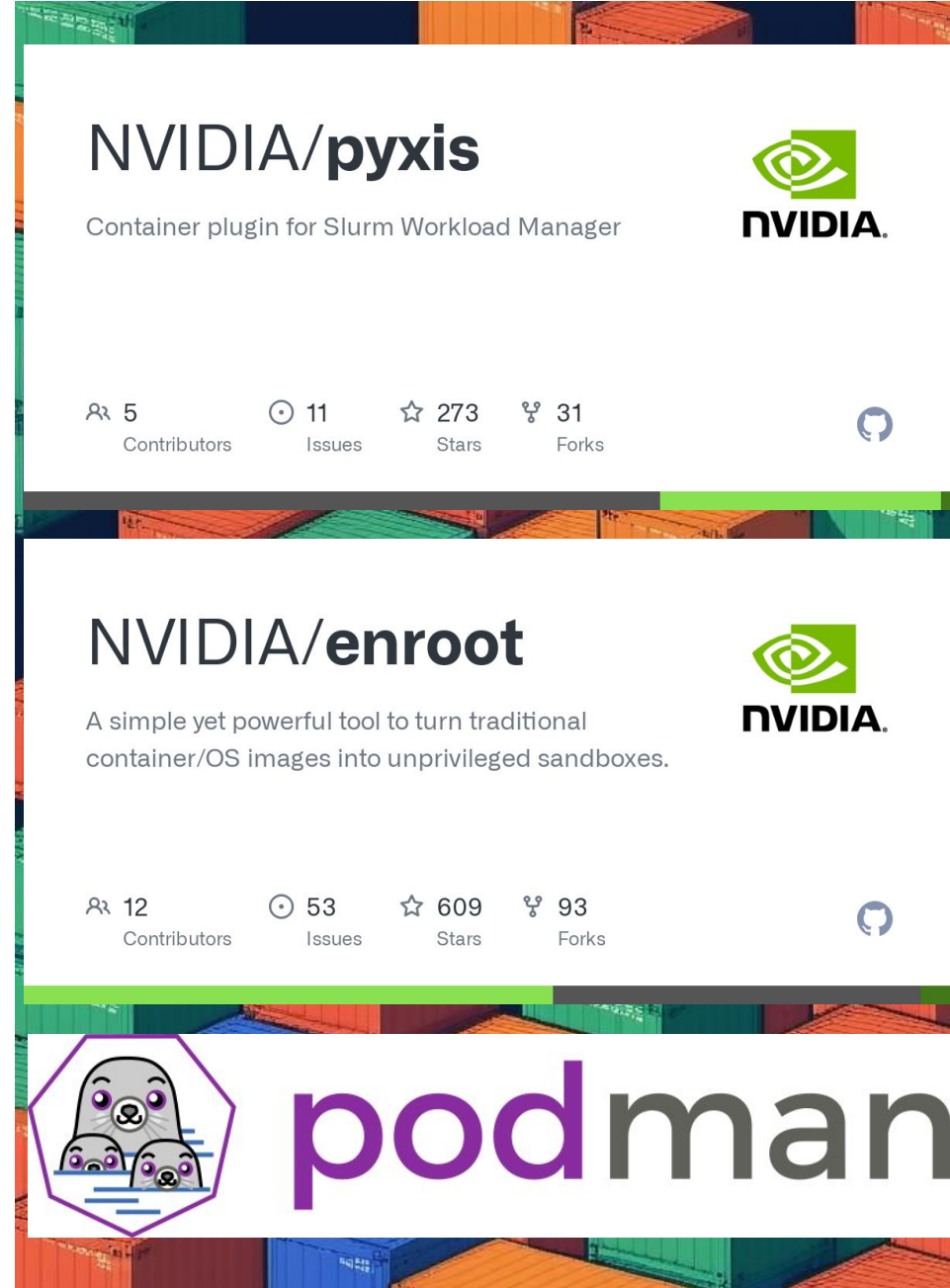
Overview

A composite Container Engine service to deploy HPC containers



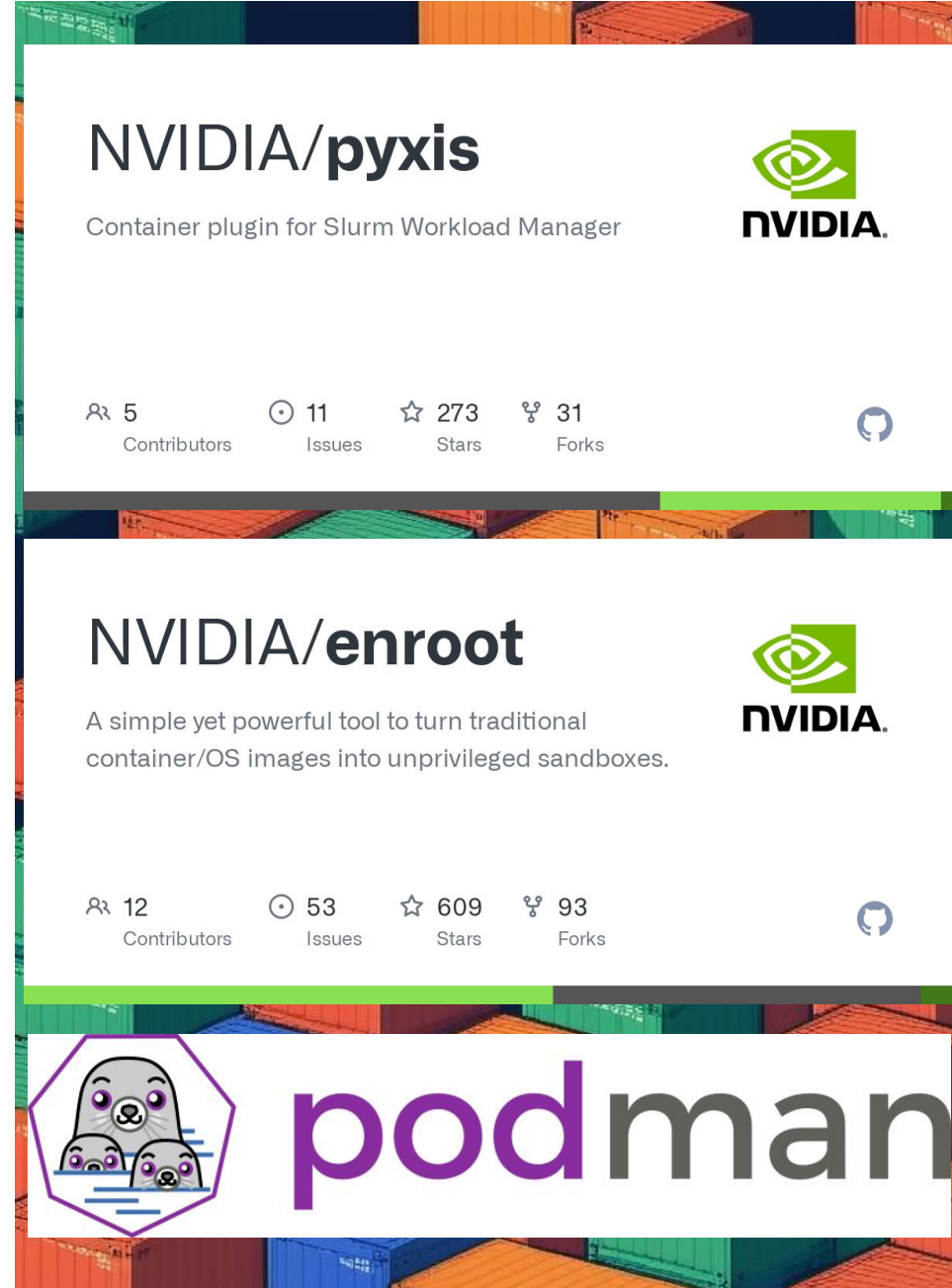
Container Engine

- Pyxis
Slurm plugin for container integration
- Enroot
Unprivileged HPC container runtime for execution at scale
- Podman
Build container images



Container Engine

- Pyxis (CSCS customized)
Slurm plugin for container integration
- Enroot (CSCS customized)
Unprivileged HPC container runtime for execution at scale
- Podman
Build container images



Container Engine - EDF: Environment Definition File

- A **TOML** file that defines image, mount points, environment variables, annotations, and more
- A blueprint to define containers as job environments
- Composable, trackable, shareable
- Still in evolution

```
image = "nvcr.io/nvidia/pytorch:26.02-py3"
mounts = [
    "/capstor:/capstor",
    "/iopsstor:/iopsstor"
]
workdir = "${SCRATCH}"

[env]
NCCL_DEBUG = "INFO"
OMP_NUM_THREADS = "72"

[annotations.com.hooks]
aws_ofi_nccl.enabled = "true"
dcgm.enabled = "true"
```


Container Engine - Quickstart

- **image** is a string representing an image reference on a registry.
It is the only strictly required field.
- **mounts** is a TOML array of strings, representing bind mounts
 - format:
“source-on-host:destination-in-container”
- **workdir** is a basic string representing initial working directory inside the container

```
[user@alps-daint]$ cat  
$HOME/.edf/ubuntu.toml
```

```
image = "library/ubuntu:24.04"
```

```
mounts = ["${SCRATCH}:${SCRATCH}"]
```

```
workdir = "${SCRATCH}"
```

```
[user@alps-daint]$ srun  
--environment=ubuntu --pty bash
```

Container Engine - Quickstart

- Containerized jobs are run through the `--environment` option to `srun`
 - The option takes an EDF name or path as value
- The Container Engine does the rest
 - Pulls and caches the image
 - Creates the container as described in the EDF
 - Moves the Slurm job inside the container
 - Slurm runs the job commands, but containerized!
- If an EDF is in the **search path** (`$HOME/.edf/` by default), it can be passed just by name
 - “ubuntu” refers to `$HOME/.edf/ubuntu.toml`
 - When using an EDF outside of the search path, enter the absolute path to the file

```
[user@alps-daint]$ cat  
$HOME/.edf/ubuntu.toml
```

```
image = "library/ubuntu:24.04"
```

```
mounts = ["${SCRATCH}:${SCRATCH}"]
```

```
workdir = "${SCRATCH}"
```

```
[user@alps-daint]$ srun  
--environment=ubuntu --pty bash
```


Container Engine - Hooks

- Containers do not normally include system-specific resources
- Hooks enable custom actions during a container's life cycle
 - Inject host libraries and generally help making optimised resources available
 - Examples: Slingshot connectivity, NCCL plugins, CUDA MPS
 - Normally enabled via [annotations](#)
 - See the [CE documentation](#)
- The CXI hook for Slingshot is enabled by default

```
[user@alps-daint]$ cat pytorch.toml

image = "nvcr.io/nvidia/pytorch:26.02-py3"
mounts = [
    "/capstor:/capstor",
    "/iopsstor:/iopsstor"
]
workdir = "${SCRATCH}"

[env]
NCCL_DEBUG = "INFO"
OMP_NUM_THREADS = "72"

[annotations.com.hooks]
aws_ofi_nccl.enabled = "true"
dcm.enabled = "true"
```

Container Engine - Building Containers

1. Configure storage locations for podman by editing `$HOME/.config/containers/storage.conf`
 - Currently we support the overlay driver and recommend `/dev/shm` as storage location
 - It is strongly recommended to build images on compute nodes

⚠ Podman images created during a job allocation are deleted when the job ends. Import images as SquashFS with the Container Engine and/or push them to a remote registry before the job allocation finishes.

2. Set up a Containerfile (a.k.a. Dockerfile)

```
[user@alps-daint]$ cat storage.conf

[storage]
  driver = "overlay"
  runroot = "/dev/shm/$USER/runroot"
  graphroot = "/dev/shm/$USER/root"
```

```
[user@alps-daint]$ cat Containerfile

FROM ubuntu:24.04

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && apt-get install -y
python3 && apt-get clean && rm -rf
/var/lib/apt/lists/*
```

Container Engine - Building Containers

4. Call Podman build to create the image

```
podman build -t <tag> -f <Containerfile> .
```

5. Push image to a remote registry

AND / OR

6. Import image in SquashFS format for execution at scale

```
enroot import -x mount [-o filename]  
podman://<reference>
```

```
[user@alps-daint]$ srun --pty bash  
[user@nid001234]$ podman build -t ubuntu-python .  
[user@nid001234]$ podman images -n  
REPOSITORY          TAG      IMAGE ID      CREATED  
SIZE  
localhost/ubuntu-python latest  3fddc294ce53  16 sec 123  
MB
```

```
[user@nid001234]$ podman login <registry_url>  
  
# Re-tag image if needed  
[user@nid001234]$ podman tag <image:tag> <registry  
url>/<image:tag>  
  
# Push the image (for docker type registries use the  
docker:// prefix)  
[user@nid001234]$ podman push [docker://]<registry  
url>/<image:tag>
```

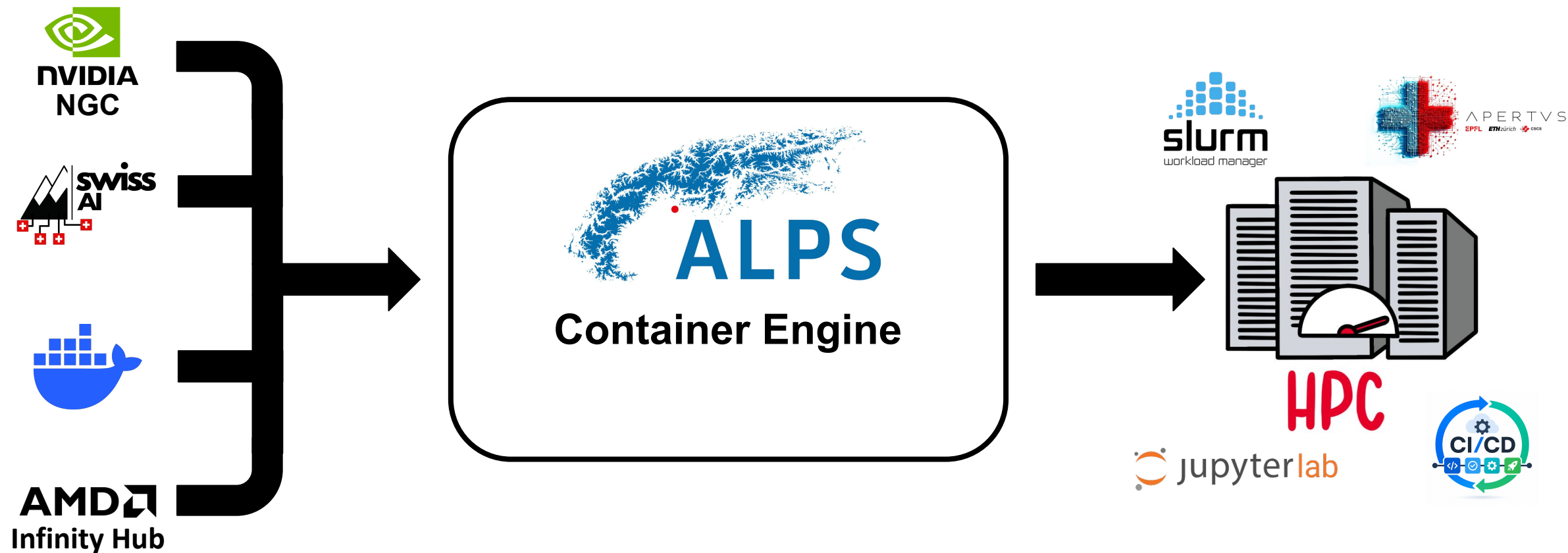
```
[user@nid001234]$ cd $SCRATCH  
[user@nid001234]$ enroot import -x mount -o  
ubuntu-python.sqsh  
podman://localhost/ubuntu-python:latest  
[user@nid001234]$ ls -l $SCRATCH/ubuntu-python.sqsh  
-rw-r--r-- 1 user group 83623936 Aug 22 15:03  
/capstor/scratch/cscs/user/ubuntu-python.sqsh  
[user@nid001234]$ exit
```

```
[user@alps-daint]$ cat ~/.edf/ubuntu-python.toml  
image = "${SCRATCH}/ubuntu-python.sqsh"
```

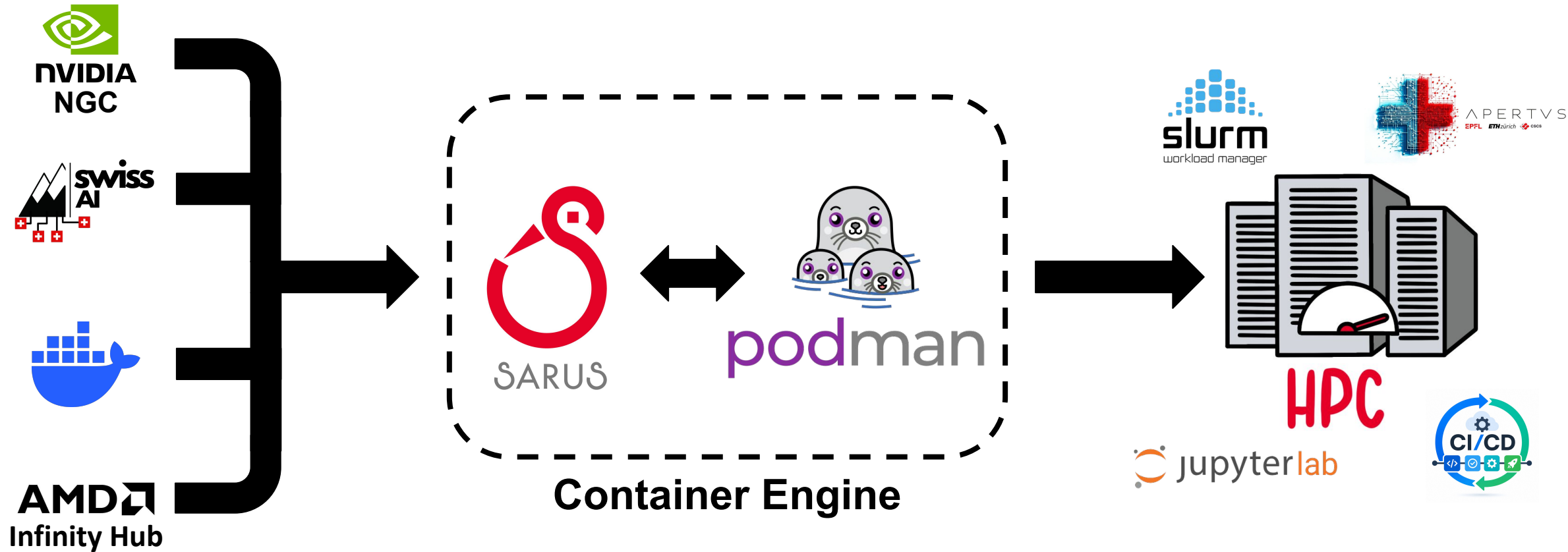
```
[user@alps-daint]$ srun -N1 -t1  
--environment=ubuntu-python python3 --version  
Python 3.12.3
```

Future outlook

A composite Container Engine service to deploy HPC containers

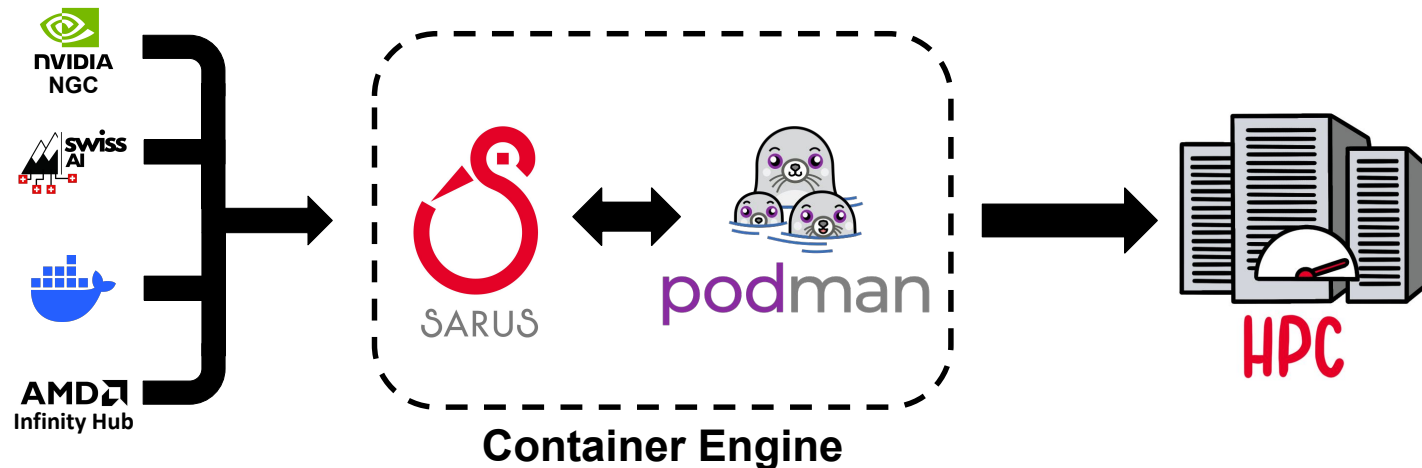


Outlook: Augmenting Podman for the next generation Container Engine



Augmenting Podman for the next generation Container Engine

- CSCS is working to promote Podman as the core runtime for the Container Engine, eventually replacing Enroot
- Podman will be integrated into the HPC infrastructure through a collection of task-specific components called **Sarus Suite**
- **Container Engine** remains as the collective HPC container offering at CSCS, maintaining a consistent user experience based on the EDF concept



Sarus Suite + Podman for the next generation Container Engine

A general purpose tool
you can run on your laptop



podman

Sarus Suite + Podman for the next generation Container Engine

A general purpose tool
you can run on your laptop



Part of the cloud native ecosystem and
compatible with popular standards



Sarus Suite + Podman for the next generation Container Engine

A general purpose tool
you can run on your laptop



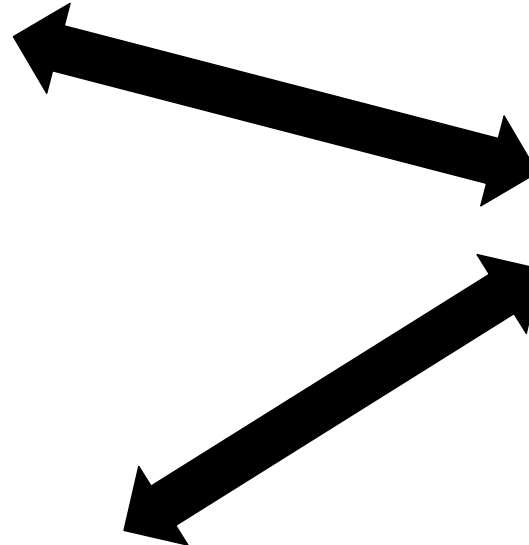
podman



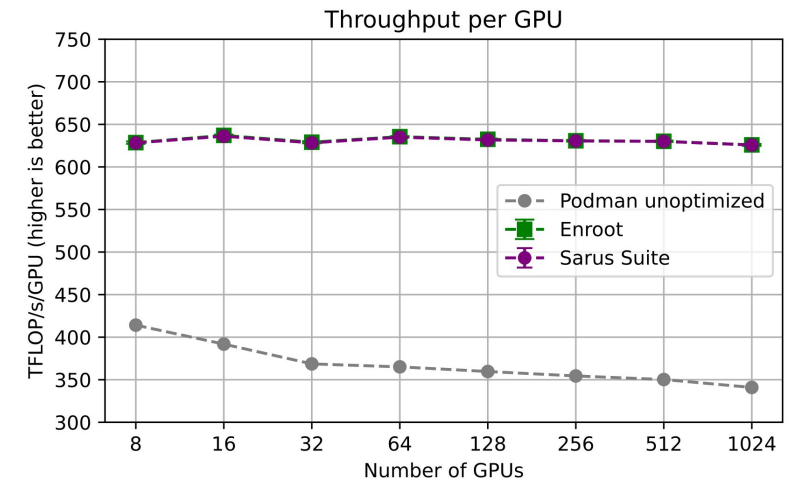
Part of the cloud native ecosystem and
compatible with popular standards



CLOUD NATIVE
COMPUTING FOUNDATION



Enabled for HPC
performance and
scalability



LLama 3 8B pre-training example with PyTorch and Megatron-LM

Some user-facing highlights

- Minimal changes to end-user experience
 - Supports EDF format and search rules
 - Uses `--edf` option instead of `--environment`
- `sarusctl` CLI tool for interactive usage
 - Parse and validate EDFs
 - Query available images
 - Run EDF-described containers interactively on login nodes for pre-flight testing
- Broader support for Bash variable expansion in EDFs

```
[user@alps-daint]$ cat ~/.edf/ubuntu.toml
image = "ubuntu:latest"

mounts = ["${SCRATCH}:${SCRATCH}"]

[env]
TEST="${NONEEXISTENT:-default}"

[user@alps-daint]$ sarusctl validate ~/.edf/ubuntu.toml
/users/<user>/~/.edf/ubuntu.toml is a valid EDF file

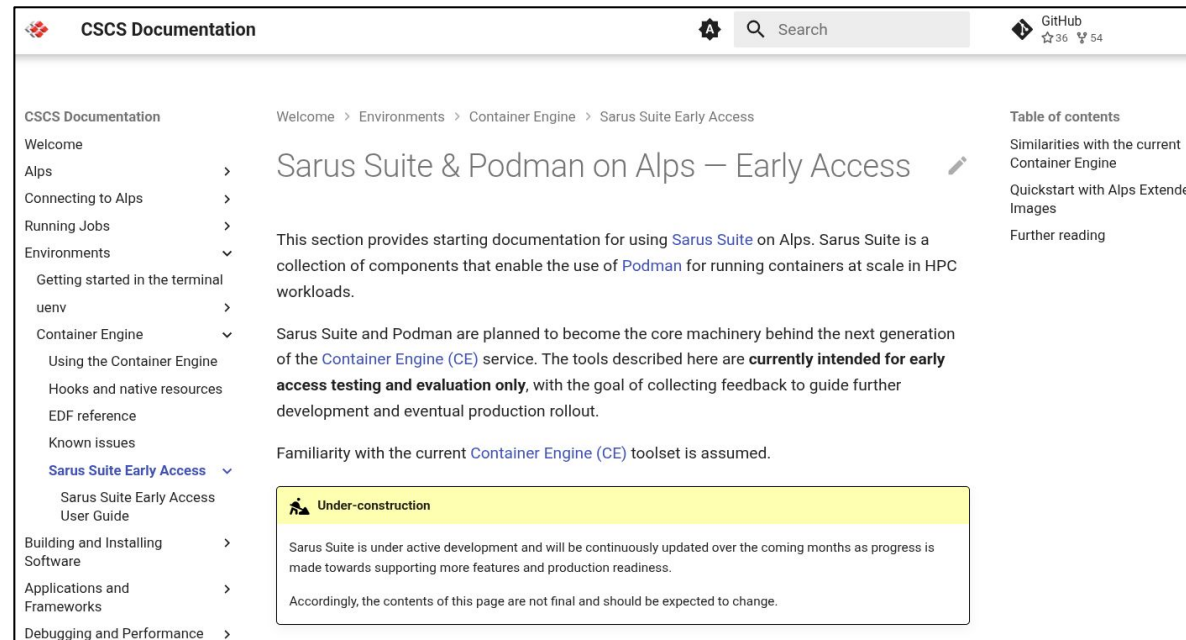
[user@alps-daint]$ sarusctl render ubuntu
{
  "annotations": {},
  "devices": [],
  "entrypoint": true,
  "env": {
    "TEST": "default"
  },
  "image": "ubuntu:latest",
  "mounts": [
    "/capstor/scratch/cscs/user:/capstor/scratch/cscs/user"
  ],
  "workdir": "",
  "writable": true
}

[user@alps-daint]$ sarusctl run ubuntu bash
user@alps-daint:/$ cat /etc/os-release | grep PRETTY
PRETTY_NAME="Ubuntu 26.04 LTS"
user@alps-daint:/$ exit

[user@alps-daint]$ srun -n2 --edf ubuntu cat /etc/issue
Ubuntu 26.04 LTS \n \l
Ubuntu 26.04 LTS \n \l
```

Sarus Suite Early Access on Alps

- Sarus Suite is already installed on Alps GH200 vClusters, but not yet complete
- We are kicking off an early access period to guide development and polish through *your* feedback
- Current CE based on Enroot & Pyxis remains as the official production solution
- A migration plan will be announced once Sarus Suite is production ready



<https://docs.cscs.ch/software/container-engine/sarus-suite/>

Sarus Suite open source project

**Modular components to unlock the power of HPC
for container tools based on open standards.**

- **Cluster Tooling**
 - Skybox Slurm plugin
 - sarusctl CLI tool
- **Performance Extensions**
 - OCI hooks, CDIs, Podman config modules
- **Transparent Storage**
 - Parallax storage tool
- **Dev & Admin Tools**
 - Deployment & automation



<https://github.com/sarus-suite>

Curious about details? Read the pre-print on ArXiv!

Computer Science > Distributed, Parallel, and Cluster Computing

[Submitted on 18 Apr 2026]

Sarus Suite: Cloud-native Containers for HPC

Alberto Madonna, Matteo Chesi, Gwangmu Lee, Michele Brambilla, Fawzi Roberto Mohamed, Felipe A. Cruz

High-performance computing (HPC) systems must support fast-moving software stacks, especially in AI/ML, while preserving scheduler control, scalable startup, and production performance. Yet many HPC container solutions rely on specialized runtime stacks that weaken continuity with mainstream cloud-native workflows and require ongoing effort to sustain compatibility with the evolving upstream ecosystem. We argue that HPC should specialize the integration layer while keeping the container engine aligned with upstream container evolution. We present Sarus Suite, an upstream-aligned HPC container architecture built around an unchanged Podman engine. Sarus Suite adds the HPC-specific functionality needed for production use through complementary system layers for declarative runtime specification, scheduler-native execution, scalable shared-image access, and standards-based host capability injection. We evaluate Sarus Suite on a Cray EX GH200 system using communication-intensive HPC workloads, large scale AI training, metadata-heavy startup workloads, and container startup measurements. Across PyFR, SPH-EXA, Megatron-LM, and Pynamic, Sarus Suite matches the performance and scaling of the production Enroot+Pyxis baseline while delivering consistently faster per-node container startup. The architecture also enables direct use of upstream OCI images, including NGC-based images, and supports cloud-native multi-container workflows expressed through Kubernetes manifests. These results show that HPC-grade containers do not require an HPC-specific runtime, provided that scheduler semantics, scalable image access, and host integration are implemented in explicit system layers. This preserves upstream continuity and software agility while maintaining scheduler control, scalability, and production performance.

Comments: 26 pages, 6 figures, 2 tables, 3 listings

Subjects: **Distributed, Parallel, and Cluster Computing (cs.DC)**

Cite as: [arXiv:2604.17064](https://arxiv.org/abs/2604.17064) [cs.DC]

(or [arXiv:2604.17064v1](https://arxiv.org/abs/2604.17064v1) [cs.DC] for this version)

<https://doi.org/10.48550/arXiv.2604.17064> 

Access Paper:

[View PDF](#)
[HTML \(experimental\)](#)
[TeX Source](#)

 [view license](#)

Current browse context:

cs.DC

[< prev](#) | [next >](#)
[new](#) | [recent](#) | [2026-04](#)

Change to browse by:

cs

References & Citations

[NASA ADS](#)
[Google Scholar](#)
[Semantic Scholar](#)

Export BibTeX Citation

Bookmark



<https://arxiv.org/abs/2604.17064>

Useful links from the CSCS Docs

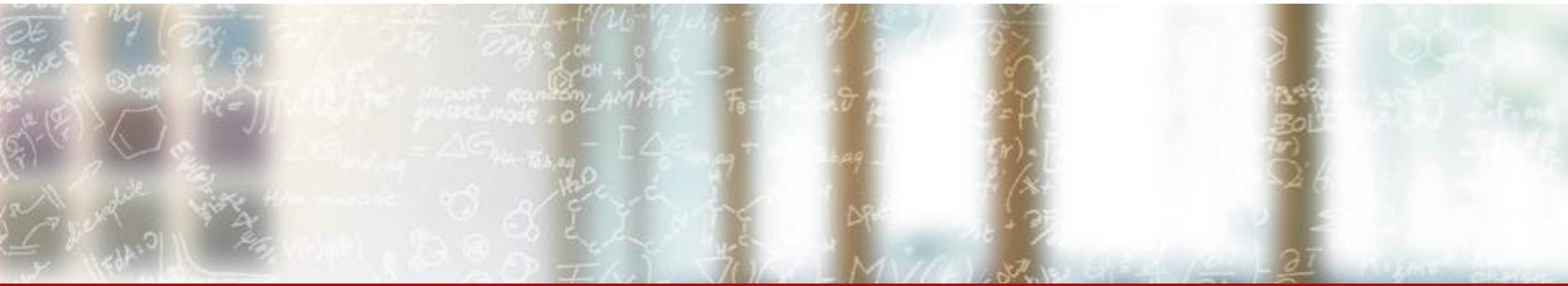
- Container Engine
 - <https://docs.cscs.ch/software/container-engine/>
- Building container images on Alps
 - <https://docs.cscs.ch/build-install/containers/#building-container-images-on-alps>
- Guidelines and snippets to work with libfabric, MPI, NCCL in containers
 - <https://docs.cscs.ch/software/communication/>
- Sarus Suite Early Access:
 - <https://docs.cscs.ch/software/container-engine/sarus-suite/>



CSCS

Centro Svizzero di Calcolo Scientifico
Swiss National Supercomputing Centre

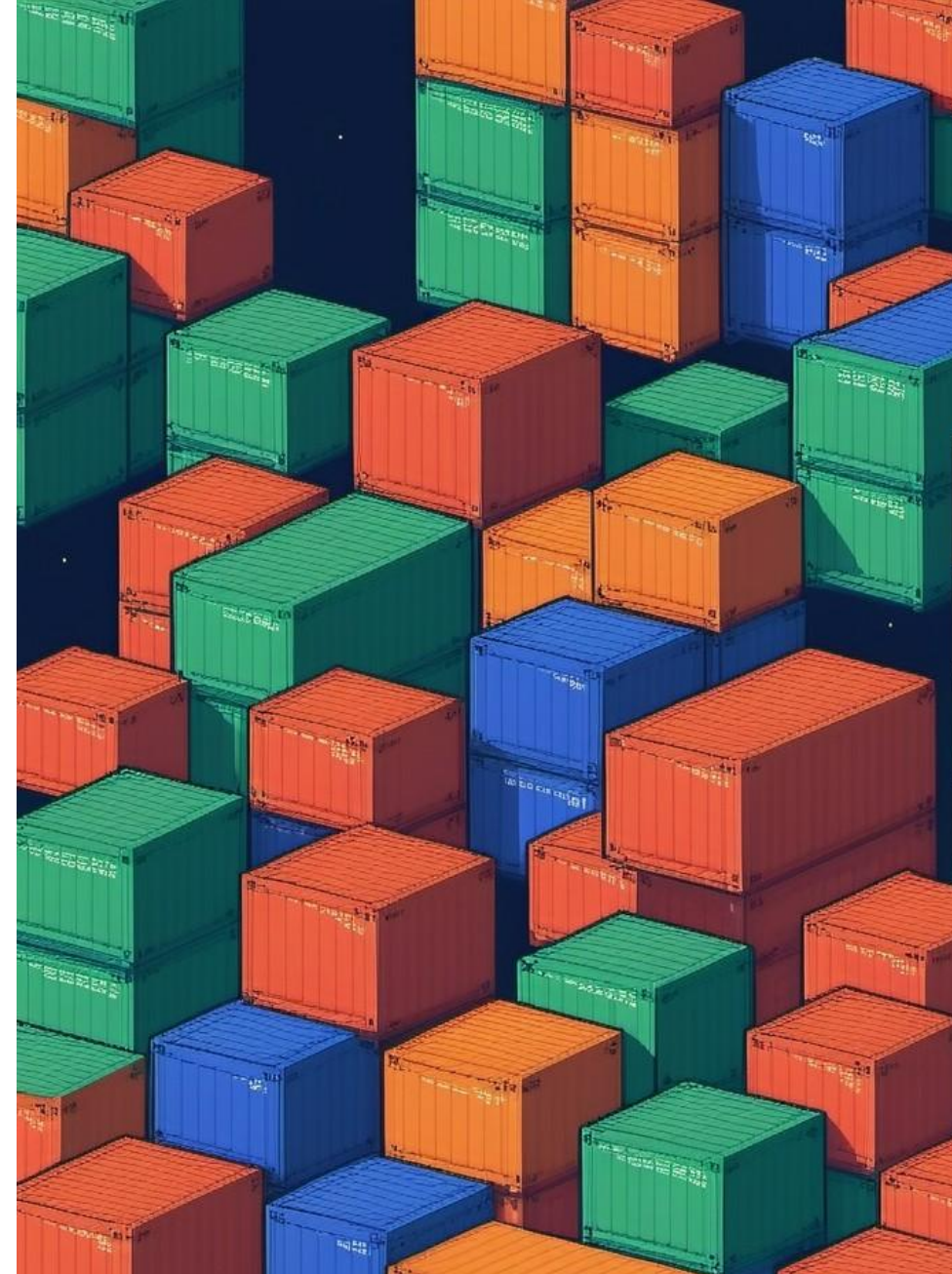
ETH zürich



Thanks for your attention!

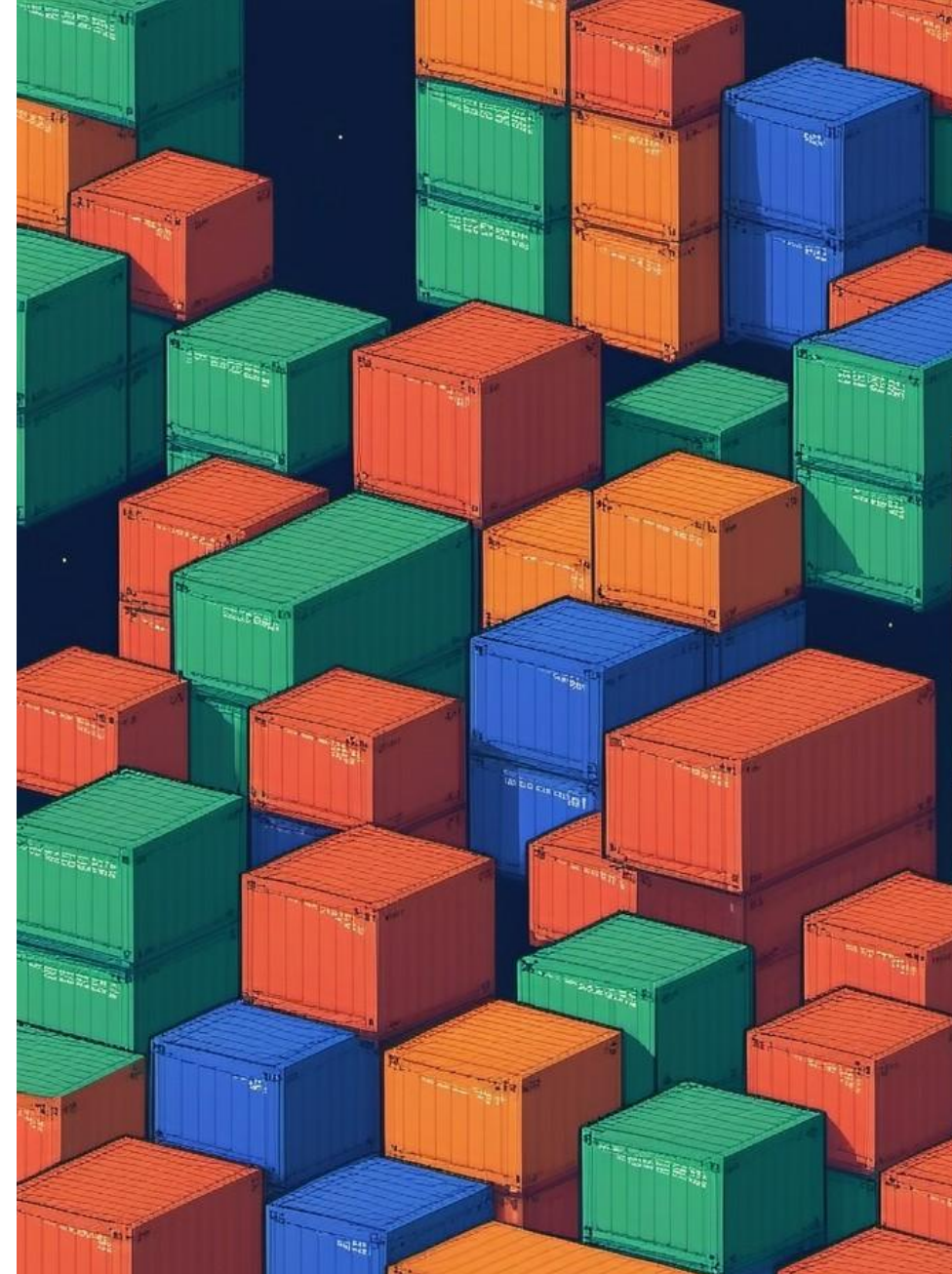
Containers

- Lightweight, standalone, (+executable) software stack that **encapsulates** an application in a single package, called an **image**
 - Code
 - Runtime + libraries
 - System tools
 - Settings
- Provide consistent environment for applications to run
- Easy to share across users/groups



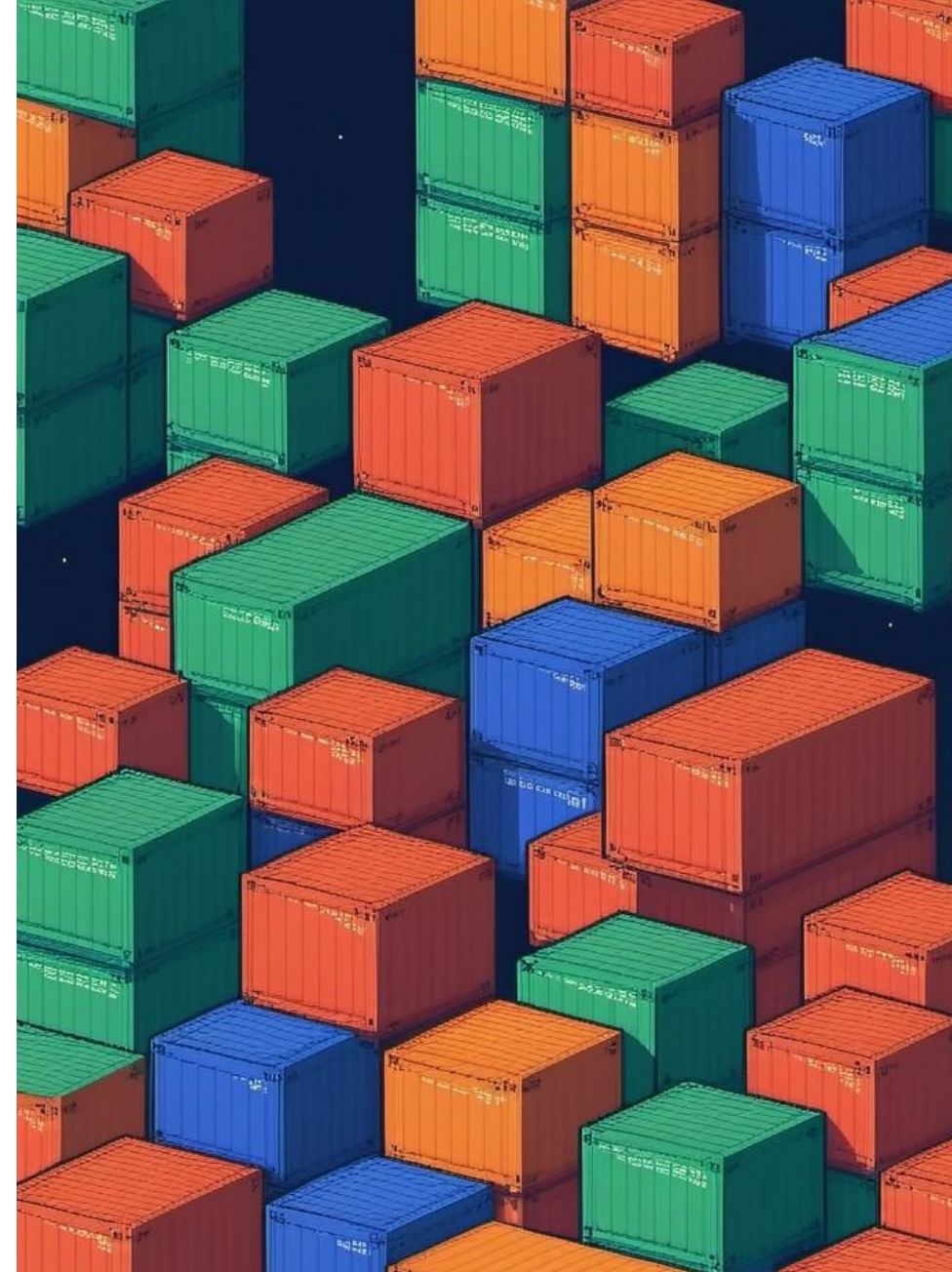
Containers

- Containers run in **isolated** namespaces, separate from the host system, though they share the same operating system kernel
- Large ecosystem of tools available
- Widely adopted
- In ML, vendors provide software stack optimised for their hardware as containers



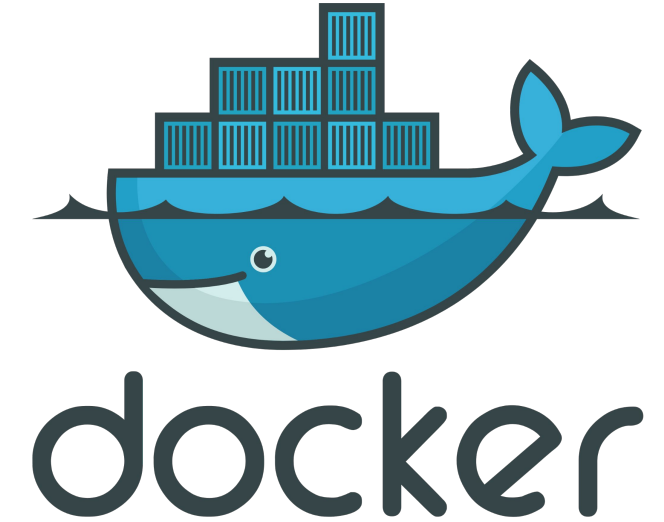
Container Images

- Container image: all binaries and libraries, replace the current OS filesystem
 - Host filesystem inaccessible unless explicitly mounted
- You can get them from registries: hub.docker.com, quay.io, catalog.ngc.nvidia.com (nvcr.io), [AMD's Infinity Hub](https://www.amd.com/en/infiniti), ...
- or build your own on HPC, cloud, laptop...



General purpose container implementations

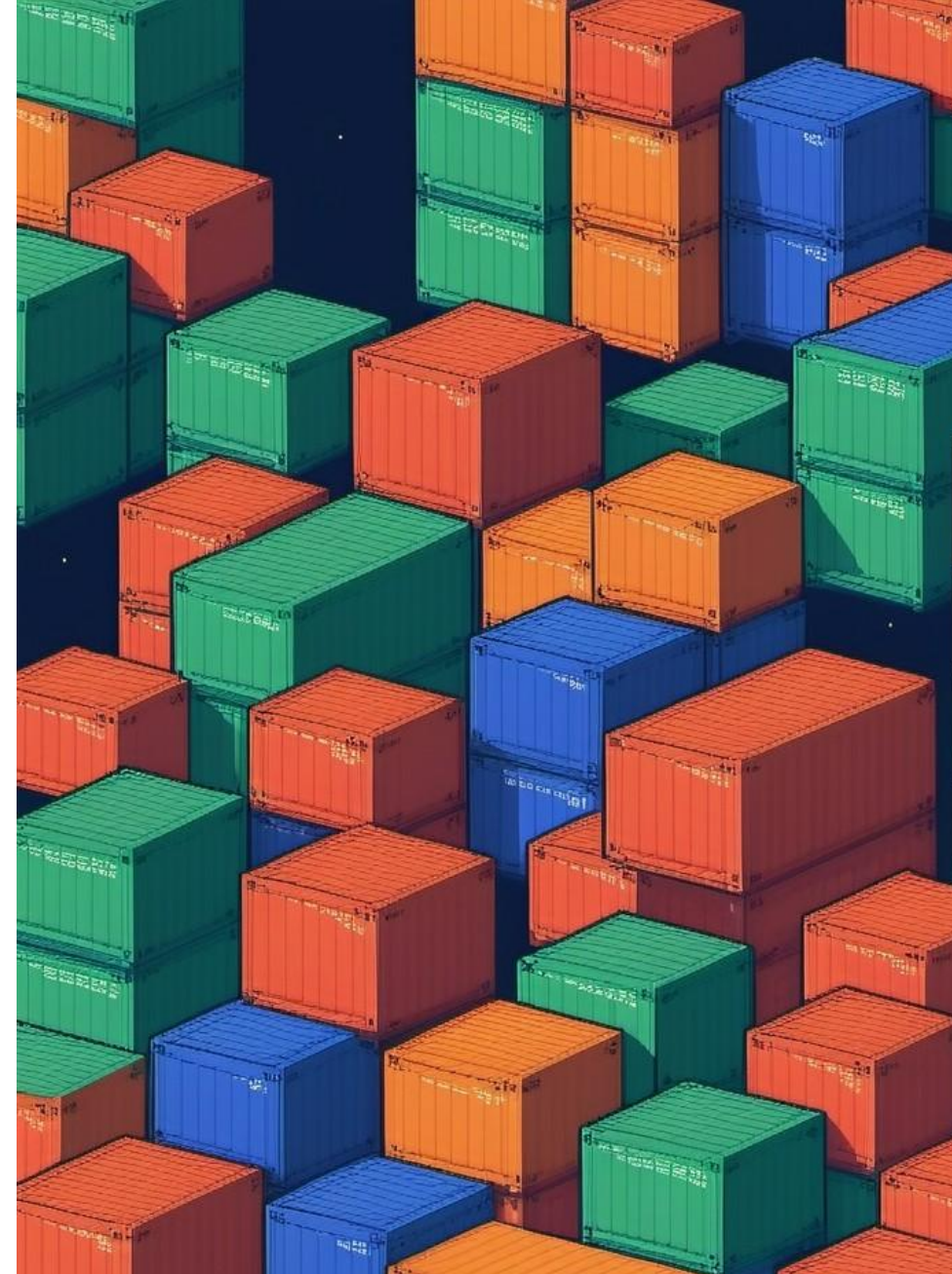
- + Widely adopted container engines
- + Robust ecosystem
- Difficulty in integrating with specialized HPC environments
 - Custom hardware (e.g. accelerators, interconnect)
 - Centralized filesystem storage
 - Distributed, shared system managed by a WLM
 - Reduced levels of isolation required (e.g. for MPI)
- Docker: Security concerns in HPC
 - Root daemon that runs your container with root privileges
- Podman: Additional complexity due to user namespaces



The Container Engine toolset

Goal: Make usage of containers more transparent and seamlessly integrated into HPC systems

- Focus on HPC: scalability and performance
- Integrated into the Slurm WLM workflow
- Modular support for HPC features (e.g. GPUs, Slingshot, NCCL) through hooks and plugins
- Declarative definition of container-based environments
- Interoperable with existing container ecosystem
- Unprivileged operation for improved security



Container Engine - Quickstart

- Best to use `--environment` as part of “srun” commands, even in sbatch scripts
- Gives possibility to switch environments/containers between job steps
- `--environment` can be used as an `#SBATCH` option, but has several pitfalls: available for corner cases and power users, otherwise **discouraged**

```
[user@alps-daint]$ cat example.sbatch
#!/bin/bash -l

#SBATCH --job-name=edf-example
#SBATCH --time=0:01:00
#SBATCH --nodes=2
#SBATCH --ntasks-per-node=1
#SBATCH --output=slurm-%x.out

# Run a simulation
srun --environment=gromacs gmx_mpi <...>

# Analyze results
srun --environment=post-proc -N1 run_post.py
```

Container Engine - Pulling Images

- Registry reference strings (library/ubuntu:22.04) results in images being cached in `$SCRATCH/.edf_imagestore/`
 - Controlled through `$EDF_IMAGESTORE` environment variable
- DockerHub is the default registry
- Also possible to pull images explicitly beforehand using Enroot
- Formats
 - `enroot import`
`docker://[REGISTRY#]IMAGE[:TAG]`
 - `enroot import docker://IMAGE[:TAG]`

```
[user@alps-daint]$ enroot import  
docker://nvidia/cuda:11.8.0-cudnn8-dev  
el-ubuntu22.04
```

```
[user@alps-daint]$ ls -l *.sqsh  
-rw-r--r-- 1 <user> csstaff 9720037376  
Sep 1 14:46  
nvidia+cuda+11.8.0-cudnn8-devel-ubuntu  
22.04.sqsh
```

NVIDIA/enroot

A simple yet powerful tool to turn traditional container/OS images into unprivileged sandboxes.



12
Contributors

53
Issues

609
Stars

93
Forks



Container Engine - Third Party or Private Repositories with Enroot

- **URL** has to be prepended to **image reference** using (#) as separator
- For private repositories, access credentials should be configured in `$HOME/.config/enroot/.credentials`
- For NGC (**nvcr.io**) private images, you'll need to register and create an API key here: catalog.ngc.nvidia.com

```
[user@alps-daint]$ enroot import  
docker://nvcr.io#nvidia/nvhpc:23.7-runtime-cuda11.8-ubuntu22.04
```

```
[user@alps-daint]$ cat  
$HOME/.config/enroot/.credentials  
  
# NVIDIA NGC catalog (both endpoints are required)  
machine nvcr.io login $oauthtoken password  
<token>  
  
machine authn.nvidia.com login $oauthtoken  
password <token>  
  
# DockerHub  
machine auth.docker.io login <login> password  
<password>  
  
# Github.com Container Registry (GITHUB_TOKEN  
needs read:packages scope)  
machine ghcr.io login <username> password  
<GITHUB_TOKEN>
```

NVIDIA GPUs

- Support for NVIDIA driver/devices is controlled by `$NVIDIA_VISIBLE_DEVICES`
- **Recommendation:** use official NVIDIA images from Docker Hub or NGC as base
 - Updated, optimized installation of CUDA Toolkit
 - Environment variables already set

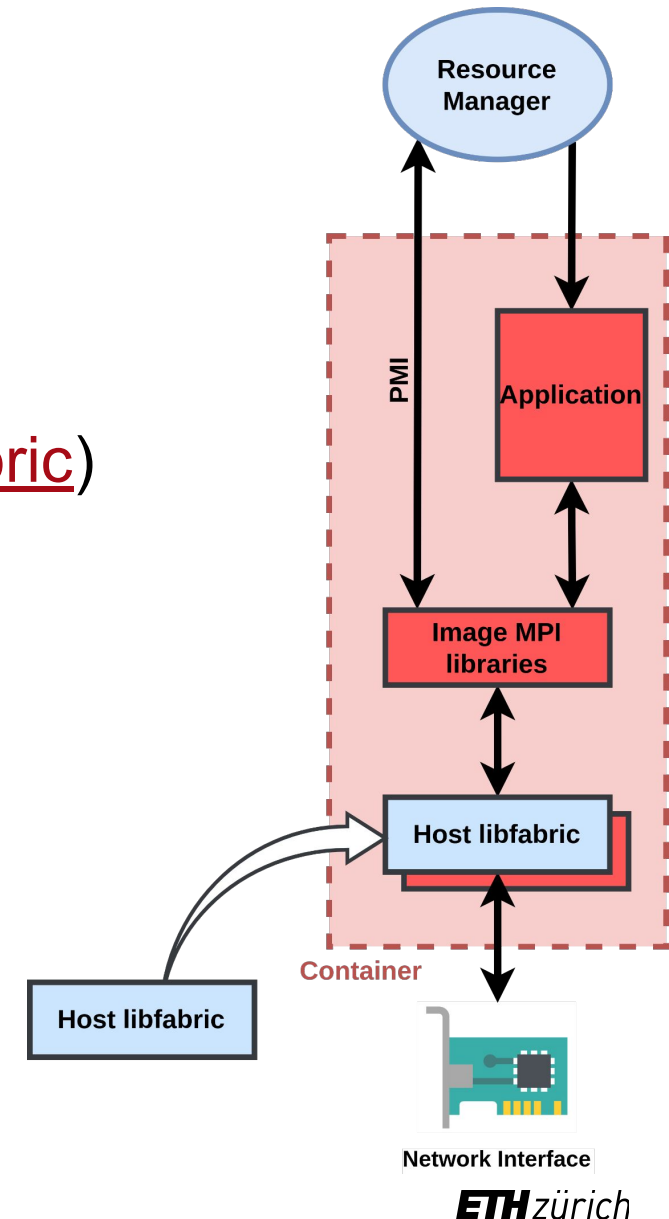
```
[user@alps-daint]$ cat $HOME/.edf/cuda.toml  
image =  
"nvidia/cuda:12.8.1-devel-ubuntu24.04"
```

```
[user@alps-daint]$ srun --environment=cuda  
nvidia-smi -L
```

```
GPU 0: NVIDIA GH200 120GB (UUID:  
GPU-3e63c1b0-b12f-7902-a89d-8d3b32f86815)  
GPU 1: NVIDIA GH200 120GB (UUID:  
GPU-d1dcf213-d242-5906-b876-91e8d8b4be3e)  
GPU 2: NVIDIA GH200 120GB (UUID:  
GPU-c2703178-3420-04eb-a5d8-4290de196468)  
GPU 3: NVIDIA GH200 120GB (UUID:  
GPU-bfcfe315-499c-4ed4-734b-8e5f1306fbc0)
```


MPI containers

- **Goal:** leverage the Slingshot high-speed network
- Recommended implementations
 - MPICH ≥ 4.1
 - OpenMPI ≥ 5
- Build MPI on top of libfabric (<https://github.com/ofiwg/libfabric>)
- The CE CXI hook (enabled by default!) replaces libfabric with a custom variant for Slingshot
 - Current version for best replacement compatibility: 1.22
- Use PMI implementation compatible with your container's MPI:
 - MPICH $\rightarrow$ PMI2: `srun --mpi=pmi2 ...`
 - OpenMPI $\rightarrow$ PMIx: `srun --mpi=pmix ...`

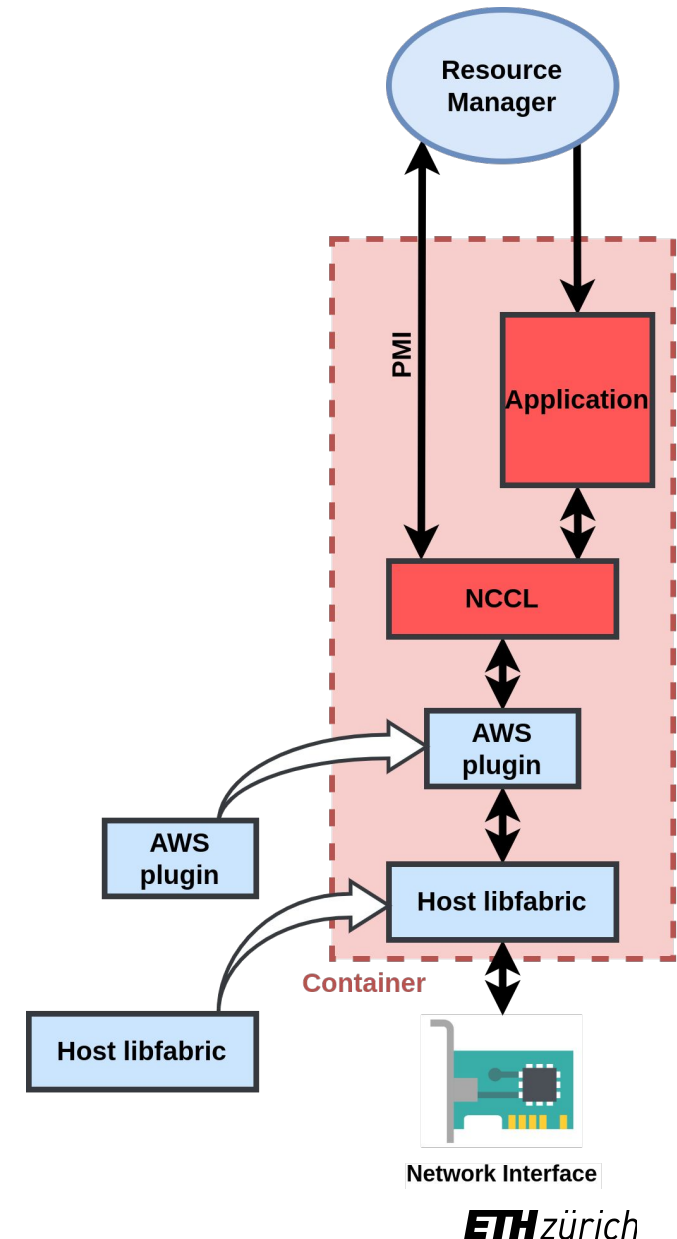


NCCL containers

- ML/AI containers predominantly rely on optimized collectives libraries (NCCL or RCCL)
- We need a way to reconcile NCCL's connection-oriented transport APIs with libfabric's connection-less interface!
- The AWS OFI NCCL hook mounts a plugin to address this

```
[annotations.com.hooks]  
aws_ofi_nccl.enabled = "true"  
aws_ofi_nccl.variant = "cuda12"
```

- For NCCL-only images (no MPI), libfabric does **not** have to be installed in the image: discovery and loading happen at runtime



Using the AWS-OFI-NCCL Hook

all reduce benchmark with PyTorch+NCCL using the 25.06-py3 PyTorch NGC image

With Hook

payload	busbw	algbw
-----:	-----:	-----:
32KiB	0.27GBps	0.14GBps
64KiB	0.54GBps	0.28GBps
128KiB	0.92GBps	0.48GBps
256KiB	1.41GBps	0.73GBps
512KiB	2.67GBps	1.38GBps
1MiB	5.27GBps	2.72GBps
2MiB	7.89GBps	4.07GBps
4MiB	10.54GBps	5.44GBps
8MiB	19.25GBps	9.93GBps
16MiB	30.67GBps	15.83GBps
32MiB	65.28GBps	33.69GBps
64MiB	84.27GBps	43.49GBps
128MiB	89.20GBps	46.04GBps
256MiB	91.06GBps	47.00GBps
512MiB	92.04GBps	47.51GBps
1GiB	92.58GBps	47.78GBps
2GiB	92.85GBps	47.92GBps
4GiB	92.98GBps	47.99GBps
8GiB	93.04GBps	48.02GBps
16GiB	92.98GBps	47.99GBps

Without hook

payload	busbw	algbw
-----:	-----:	-----:
32KiB	0.08GBps	0.04GBps
64KiB	0.17GBps	0.09GBps
128KiB	0.31GBps	0.16GBps
256KiB	0.37GBps	0.19GBps
512KiB	0.42GBps	0.22GBps
1MiB	1.19GBps	0.61GBps
2MiB	1.62GBps	0.84GBps
4MiB	2.10GBps	1.08GBps
8MiB	2.85GBps	1.47GBps
16MiB	2.98GBps	1.54GBps
32MiB	2.65GBps	1.37GBps
64MiB	2.80GBps	1.45GBps
128MiB	2.97GBps	1.53GBps
256MiB	3.08GBps	1.59GBps
512MiB	3.23GBps	1.67GBps
1GiB	3.14GBps	1.62GBps
2GiB	3.15GBps	1.63GBps
4GiB	3.19GBps	1.65GBps
8GiB	3.21GBps	1.65GBps
16GiB	3.20GBps	1.65GBps